

Отладка корутин

При использовании stackful-корутин [для исполнения методов БЛ](#) в облачных и настольных решениях, для [синхронизации данных](#) в автономных приложениях рано или поздно возникает необходимость отладки исходного кода, исполняемого в корутине. Если отладка активной корутины не представляет никаких трудностей и ничем не отличается от отладки потока исполнения, изучение состояния приостановленной корутины, напротив, довольно проблематично. Это неудобство наступает в силу того, что отладчик ничего не знает о применяемых нами нестандартных корутинах и, как следствие, не представляет их в удобном для исследования формате.

Для заполнения этой бреши платформа предоставляет скриптовые расширения для отладчиков GDB и LLDB, позволяющие исследовать состояние приостановленных корутин как исполняющегося сервиса или приложения, так и post-mortem (т.е. дампов памяти процессов). Расширения поставляются через [SBIS SDK](#), автоматически прописываются инсталлятором в конфигурационных файлах поддерживаемых отладчиков и доступны в каждой отладочной сессии.

В силу различий GDB и LLDB как по множеству и синтаксису команд, так и в программных интерфейсах, расширения отладки корутин для каждого поддерживаемого отладчика индивидуальны. Однако в обоих случаях для изучения состояния корутин потребуется лишь консоль отладчика, предоставляемая непосредственно командой gdb или lldb, либо интегрированная в IDE (Debugger Log в Qt Creator и Debug Console в Visual Studio Code).

Отладка в GDB

Этот раздел описывает отладку корутинизированных облачных сервисов и настольных решений для платформы Linux и других Unix-подобных ОС.

Идентификация потока-исполнителя корутин

В силу архитектурных ограничений расширение GDB способно работать только с корутинами, исполняемыми выбранным потоком. Поэтому сначала следует идентифицировать поток планировщика корутин. Индикатором такого потока служит наличие на стеке вызовов кадра sbis::fiber::Scheduler::Schedule или sbis::fiber::Fiber::Run.

```
Файл Правка Вид Поиск Терминал Справка
Thread 22 (Thread 0x7f9396dfb700 (LWP 12461)):
#0 0x00007f93c9973de2 in pthread_cond_timedwait@@GLIBC_2.3.2 () at /lib64/libpthread.so.0
#1 0x00007f93c7a52102 in __gthread_cond_timedwait(pthread_cond_t*, pthread_mutex_t*, timespec const*) (_cond=0x7f9396dfac90, __mutex=0x7f9396dfac60, __abs_timeout=0x7f9396dfa800) at /opt/gcc-9.1.1/lib/gcc/x86_64-redhat-linux/9/../../../../include/c++/9/x86_64-redhat-linux/bits/gthr-default.h:872
#2 0x00007f93c7a51e06 in std::condition_variable::_wait_until_impl<std::chrono::duration<long, std::ratio<11, 1000000000000> > >(<std::unique_lock<std::mutex>&, std::chrono::time_point<std::chrono::V2::steady_clock, std::chrono::duration<long, std::ratio<11, 1000000000000> > > const&, (this=0x7f9396dfac90, __lock=..., __atime=...) at /opt/gcc-9.1.1/lib/gcc/x86_64-redhat-linux/9/../../../../include/c++/9/condition_variable:188
#3 0x00007f93c7a51c5e in std::condition_variable::wait_until<std::chrono::V2::steady_clock, std::chrono::duration<long, std::ratio<11, 1000000000000> > >(<std::unique_lock<std::mutex>&, std::chrono::time_point<std::chrono::V2::steady_clock, std::chrono::duration<long, std::ratio<11, 1000000000000> > > const&, (this=0x7f9396dfac90, __lock=..., __atime=...) at /opt/gcc-9.1.1/lib/gcc/x86_64-redhat-linux/9/../../../../include/c++/9/condition_variable:121
#4 0x00007f93c7a51ab6 in std::condition_variable::wait_until<std::chrono::V2::steady_clock, std::chrono::duration<long, std::ratio<11, 1000000000000> > >, sbis::Condition::WaitUntil(sbis::DateTime const&):$ 1>(<std::unique_lock<std::mutex>&, std::chrono::time_point<std::chrono::V2::steady_clock, std::chrono::duration<long, std::ratio<11, 1000000000000> > > const&, sbis::Condition::WaitUntil(sbis::DateTime const&):$ 1) (this=0x7f9396dfac90, __lock=..., __atime=..., __p=...) at /opt/gcc-9.1.1/lib/gcc/x86_64-redhat-linux/9/../../../../include/c++/9/condition_variable:138
#5 0x00007f93c7a3b2ab in std::condition_variable::wait_for<long, std::ratio<11, 1000000000000>, sbis::Condition::WaitUntil(sbis::DateTime const&):$ 1>(<std::unique_lock<std::mutex>&, std::chrono::duration<long, std::ratio<11, 1000000000000> > const&, sbis::Condition::WaitUntil(sbis::DateTime const&):$ 1) (this=0x7f9396dfac90, __lock=..., __rtime=..., __p=...) at /opt/gcc-9.1.1/lib/gcc/x86_64-redhat-linux/9/../../../../include/c++/9/condition_variable:165
#6 0x00007f93c7a3b1a0 in sbis::Condition::WaitUntil(sbis::DateTime const&) (this=0x7f9396dfac20, timeout_time=...) at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-lib/implementation/types/future/condition.cpp:55
#7 0x00007f93c7a3fcd1 in sbis::Waiter::WaitUntil(sbis::DateTime const&) (this=0x7f9396dfabe0, timeout_time=...) at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-lib/implementation/types/future/wait.cpp:89
#8 0x00007f93c9c3fc7c in sbis::fiber::OwningScheduler::WaitReadyFibers(sbis::Waiter&) (this=0x7f93a801ce60, w=...) at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-fiber/implementation/scheduler.cpp:229
#9 0x00007f93c9c3faf8 in sbis::fiber::OwningScheduler::WaitForReadyFiber() (this=0x7f93a801ce60) at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-fiber/implementation/scheduler.cpp:210
#10 0x00007f93c9c3e447 in sbis::fiber::Scheduler::Schedule() (this=0x7f93a801ce60) at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-fiber/implementation/scheduler.cpp:99
#11 0x00007f93c8992b4a in sbis::rpc::ThreadExecutorPool::ThreadFuncProxy() (this=0x28f4f50) at /home/local/REGION/en.badaev/sdk/core/core-sever/sbis-rpc-service/implementation/process_pool/thread_executor_pool.cpp:389
```

Перечисление корутин в потоке

Для вывода корутин, исполняемых в требуемом потоке, необходимо выбрать поток в качестве текущего командой thread <индекс-потока> и вызвать команду info fibers.

В консоль будут выведены: индекс корутины, адрес объекта корутины (sbis::fiber::Fiber), статус корутины (Active, Busy, Idle) и актуальная позиция в стеке вызовов (адрес инструкции, имя функции, позиция в исходном коде).

```

Файл Правка Вид Поиск Терминал Справка
(gdb) thread 22
[Switching to thread 22 (Thread 0x7f9396dfb700 (LWP 1246))]
#0 0x00007f93c9973de2 in pthread_cond_timedwait@@GLIBC_2.3.2 () from /lib64/libpthread.so.0
(gdb) info fibers
1 Fiber 0x7f93a8029d50 (Busy) #5 0x00007f93c7a1fb96 in sbis::YieldFiber(sbis::Waitable&, sbis::DateTime const&) at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-lib/implementation/types/future/yield.cpp:37
2 Fiber 0x7f93a8030160 (Idle) #4 0x00007f93c7a1fb20 in sbis::YieldFiber(sbis::Waitable&) at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-lib/implementation/types/future/yield.cpp:30
(gdb)

```

Исследование состояния корутины

Для изучения состояния конкретной корутины требуется переключить отладчик в режим работы с корутиной командой `fiber <индекс корутины>`, где в качестве индекса корутины указывается одно из значений выданное командой `info fibers` на предыдущем шаге.

После переключения отладчика в режим работы с корутиной все его команды исследования стека вызовов выводят данные выбранной корутины (тогда как в штатном режиме они выводят данные выбранного потока).

Команда `backtrace` выводит стек вызовов корутины.

```

Файл Правка Вид Поиск Терминал Справка
(gdb) fiber 1
[Switching to fiber 1 (Fiber 0x7f93a8029d50 (Busy))]
(gdb) backtrace
#1 0x00007f93c9c383c8 in boost::context::fiber::resume() && (this=0x7f93a8029da8)
    at /home/local/REGION/en.badaev/.conan_data/boost/rc-23.5000-56/sbis/stable/package/dac0ed37c6e6c9fa41750b7632b4c3674881d672/include/boost/context/fiber_fcontext.hpp:279
#2 0x00007f93c9c38af8 in sbis::fiber::Fiber::YieldFiber() (this=0x7f93a8029d50)
    at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-fiber/implementation/fiber.cpp:520
#3 0x00007f93c9c38c65 in sbis::fiber::Fiber::YieldFiber(sbis::Waitable&) (this=0x7f93a8029d50, w=...)
    at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-fiber/implementation/fiber.cpp:533
#4 0x00007f93c9c38e4b in sbis::fiber::Fiber::YieldFiber(sbis::Waitable&, sbis::DateTime) (this=0x7f93a8029d50, w=..., until=...)
    at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-fiber/implementation/fiber.cpp:547
#5 0x00007f93c9c56c07 in sbis::fiber::(anonymous namespace)::YieldReactor::YieldFiber(sbis::Waitable&, sbis::DateTime const&)
    (this=0x7f93c9c6c028 <sbis::sbis_fiber300_initializers::InitFiberFactory()::REG+32>, waitable=..., until=...)
    at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-fiber/implementation/fiber.cpp:643
#6 0x00007f93c7a1fb96 in sbis::YieldFiber(sbis::Waitable&, sbis::DateTime const&) (waitable=..., until=...)
    at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-lib/implementation/types/future/yield.cpp:37
#7 0x00007f93c7a3b109 in sbis::Condition::WaitUntil(sbis::DateTime const&) (this=0x7f93a802a210, timeout_time=...)
    at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-lib/implementation/types/future/condition.cpp:52
#8 0x00007f93c6baee8 in sbis::dls::UnlockEvent::WaitUntil(sbis::DateTime) (this=0x7f93a802a210, timeout_time=...)
    at /home/local/REGION/en.badaev/middleware/services/sbis-distributed-locks-service/implementation/lock_events.cpp:45
#9 0x00007f93c6bd2b2 in sbis::dls::LockManager::AcquireLock(sbis::dls::LockKind, std::::__cxx11::basic_string<wchar_t, std::char_traits<wchar_t>, std::allocator<wchar_t>> const&, std::::__cxx11::basic_string<wchar_t, std::char_traits<wchar_t>, std::allocator<wchar_t>> const&, boost::uuids::uuid const&, std::optional<int>, std::::__cxx11::basic_string<wchar_t, std::char_traits<wchar_t>, std::allocator<wchar_t>> const&, boost::uuids::uuid&, std::optional<int>> const&, std::::__cxx11::basic_string<wchar_t, std::char_traits<wchar_t>, std::allocator<wchar_t>> const&)
    [kind=sbis::dls::LockKind::Read, class=L"Test", key=L"Lock", lock_id=(boost::uuids::uuid) fedcba98-7654-3210-0123-456789abcdec, timeout_ms=std::optional<int> = {...}, host=L"AB", reason=L""]
    at /home/local/REGION/en.badaev/middleware/services/sbis-distributed-locks-service/implementation/lock_manager.cpp:269
#10 0x00007f93c6cb171 in sbis::dls::DistributedLocks_AcquireReadLock(sbis::EnumInfo&, std::::__cxx11::basic_string<wchar_t, std::char_traits<wchar_t>, std::allocator<wchar_t>> &, std::::__cxx11::basic_string<wchar_t, std::char_traits<wchar_t>, std::allocator<wchar_t>> &, std::::__cxx11::basic_string<wchar_t, std::char_traits<wchar_t>, std::allocator<wchar_t>> &, boost::uuids::uuid&, std::optional<int>> &)
    (result=..., host=L"AB", class=L"Test", key=L"Lock", lock_id=(boost::uuids::uuid) fedcba98-7654-3210-0123-456789abcdec, timeout_ms=std::optional<int> = {...})
    at /home/local/REGION/en.badaev/middleware/services/sbis-distributed-locks-service/implementation/lock_methods.cpp:95

```

Изучить отдельный кадр стека и его переменные можно с помощью команд `frame <индекс кадра>`, `info args`, `info locals`.

```

Файл Правка Вид Поиск Терминал Справка
(gdb) frame 9
#9 0x00007f93c6bd2b2 in sbis::dls::LockManager::AcquireLock (kind=sbis::dls::LockKind::Read, class=L"Test", key=L"Lock", lock_id=(boost::uuids::uuid) fedcba98-7654-3210-0123-456789abcdec, timeout_ms=std::optional<int> = {...}, host=L"AB", reason=L"")
    at /home/local/REGION/en.badaev/middleware/services/sbis-distributed-locks-service/implementation/lock_manager.cpp:269
269 if( !timeout || !unlock_event->WaitUntil( *timeout ) )
(gdb) info args
kind = sbis::dls::LockKind::Read
class = L"Test"
key = L"Lock"
lock_id = (boost::uuids::uuid) fedcba98-7654-3210-0123-456789abcdec
timeout_ms = std::optional<int> = {[contained value] = {_M_empty = {<No data fields>}, _M_value = 5000}}
host = L"AB"
reason = L""
(gdb) info locals
sbis::dls::LockManager::Status::Acquired = sbis::dls::LockManager::Status::Acquired
sbis::dls::LockManager::Status::Busy = sbis::dls::LockManager::Status::Busy
sbis::dls::LockManager::Status::ClaimPending = sbis::dls::LockManager::Status::ClaimPending
sbis::dls::LockManager::Status::Collision = sbis::dls::LockManager::Status::Collision
timeout = std::optional<sbis::DateTime> = {[contained value] = {_M_empty = {<No data fields>}, _M_value = {
    mDateTime = {boost::posix_time::ptime} 2023-07-18 12:05:10.1558592)}}
resource_id = L"Test.Lock"
lock_timeout_ms = 60000
revocation_timeout_ms = 10000
wait_timeout_ms = 0
script_args = {_M_elems = {L"Read", L"Test", L"Lock", L"fedcba98-7654-3210-0123-456789abcdec", L"AB", L"", L"60000", L"60000", L"0"}}
unlock_event = warning: RTTI symbol not found for class 'std::_Sp_counted_ptr_inplace<sbis::dls::UnlockEvent, std::allocator<sbis::dls::UnlockEvent>, (_gnu_cxx::Lock_policy)2>'
warning: RTTI symbol not found for class 'std::_Sp_counted_ptr_inplace<sbis::dls::UnlockEvent, std::allocator<sbis::dls::UnlockEvent>, (_gnu_cxx::Lock_policy)2>'
std::shared_ptr<sbis::dls::UnlockEvent> (use count 1, weak count 1) = {get() = 0x7f93a802a210}
status = sbis::dls::LockManager::Status::Busy
(gdb)

```

По окончании исследования состояния корутин(ы), если необходимо продолжить отладку текущего или иного потока - следует вывести GDB из режима работы с корутиной командой `fiber deactivate`.

Вспомогательные команды

Вывод команды `info fibers` можно подстроить под нужды текущей сессии отладки путем изменения множества ключевых слов, используемых для пропуска неактуальных кадров стека вызовов. По умолчанию с вершины стека отбрасываются кадры пространств имен `boost::context` и `sbis::fiber` и в качестве актуальной позиции выводятся данные первого встреченного кадра не из этих пространств.

Настройка списка фильтров осуществляется командой `set fiber info-frame-skip-words [<слово> [<слово> [...]]`, где слово -- любая комбинация символов без пробелов. Вывести текущий состав фильтров можно командой `show fiber`.

```
Файл Правка Вид Поиск Терминал Справка
(gdb) show fiber
info-frame-skip-words: boost::context sbis::fiber
(gdb) set fiber info-frame-skip-words
(gdb) info fibers
1 Fiber 0x7f93a8029d50 (Busy) #1 0x00007f93c9c383c8 in boost::context::fiber::resume() && at /home/local/REGION/en.badaev/.conan_data/boost/rc-23.5000-56/sbis/stable/package/dac0ed37c6e6c9fa41750b7632b4c3674881d672/include/boost/context/fiber_fcontext.hpp:279
2 Fiber 0x7f93a8030160 (Idle) #1 0x00007f93c9c383c8 in boost::context::fiber::resume() && at /home/local/REGION/en.badaev/.conan_data/boost/rc-23.5000-56/sbis/stable/package/dac0ed37c6e6c9fa41750b7632b4c3674881d672/include/boost/context/fiber_fcontext.hpp:279
(gdb)
```

Если необходимо применить некоторую команду ко всем корутинам выбранного потока, можно воспользоваться командой `fiber apply all <command>` (например, `fiber apply all backtrace` выведет стек вызова всех корутин).

В качестве альтернативы команда может быть исполнена лишь на некотором подмножестве корутин. Для этого вместо ключевого слова `all` следует указать список индексов целевых корутин (к примеру, `fiber apply 1 3 5 backtrace` выведет стек вызова корутин с индексами 1, 3, 5).

```
Файл Правка Вид Поиск Терминал Справка
(gdb) fiber apply all backtrace 5
Fiber 1 (Fiber 0x7f93a8029d50 (Busy))
#1 0x00007f93c9c383c8 in boost::context::fiber::resume() && (this=0x7f93a8029d50)
   at /home/local/REGION/en.badaev/.conan_data/boost/rc-23.5000-56/sbis/stable/package/dac0ed37c6e6c9fa41750b7632b4c3674881d672/include/boost/context/fiber_fcontext.hpp:279
#2 0x00007f93c9c38af8 in sbis::fiber::Fiber::YieldFiber() (this=0x7f93a8029d50)
   at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-fiber/implementation/fiber.cpp:520
#3 0x00007f93c9c38c65 in sbis::fiber::Fiber::YieldFiber(sbis::Waitable&) (this=0x7f93a8029d50, w=...)
   at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-fiber/implementation/fiber.cpp:533
#4 0x00007f93c9c38e4b in sbis::fiber::Fiber::YieldFiber(sbis::Waitable&, sbis::DateTime) (this=0x7f93a8029d50, w=..., until=...)
   at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-fiber/implementation/fiber.cpp:547
#5 0x00007f93c9c56c07 in sbis::fiber::(anonymous namespace)::YieldReactor::YieldFiber(sbis::Waitable&, sbis::DateTime const&)
   (this=0x7f93c9c6e028 <sbis::sbis_fiber300_initializers::InitFiberFactory():REG+32>, waitable=..., until=...)
   at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-fiber/implementation/fiber.cpp:643
(More stack frames follow...)
Fiber 2 (Fiber 0x7f93a8030160 (Idle))
#1 0x00007f93c9c383c8 in boost::context::fiber::resume() && (this=0x7f93a8030160)
   at /home/local/REGION/en.badaev/.conan_data/boost/rc-23.5000-56/sbis/stable/package/dac0ed37c6e6c9fa41750b7632b4c3674881d672/include/boost/context/fiber_fcontext.hpp:279
#2 0x00007f93c9c38af8 in sbis::fiber::Fiber::YieldFiber() (this=0x7f93a8030160)
   at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-fiber/implementation/fiber.cpp:520
#3 0x00007f93c9c38c65 in sbis::fiber::Fiber::YieldFiber(sbis::Waitable&) (this=0x7f93a8030160, w=...)
   at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-fiber/implementation/fiber.cpp:533
#4 0x00007f93c9c56b74 in sbis::fiber::(anonymous namespace)::YieldReactor::YieldFiber(sbis::Waitable&)
   (this=0x7f93c9c6e028 <sbis::sbis_fiber300_initializers::InitFiberFactory():REG+32>, waitable=...)
   at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-fiber/implementation/fiber.cpp:636
#5 0x00007f93c7a1fb2e in sbis::YieldFiber(sbis::Waitable&) (waitable=...)
   at /home/local/REGION/en.badaev/sdk/core/core-common/sbis-lib/implementation/types/future/yield.cpp:30
(More stack frames follow...)
(gdb) █
```

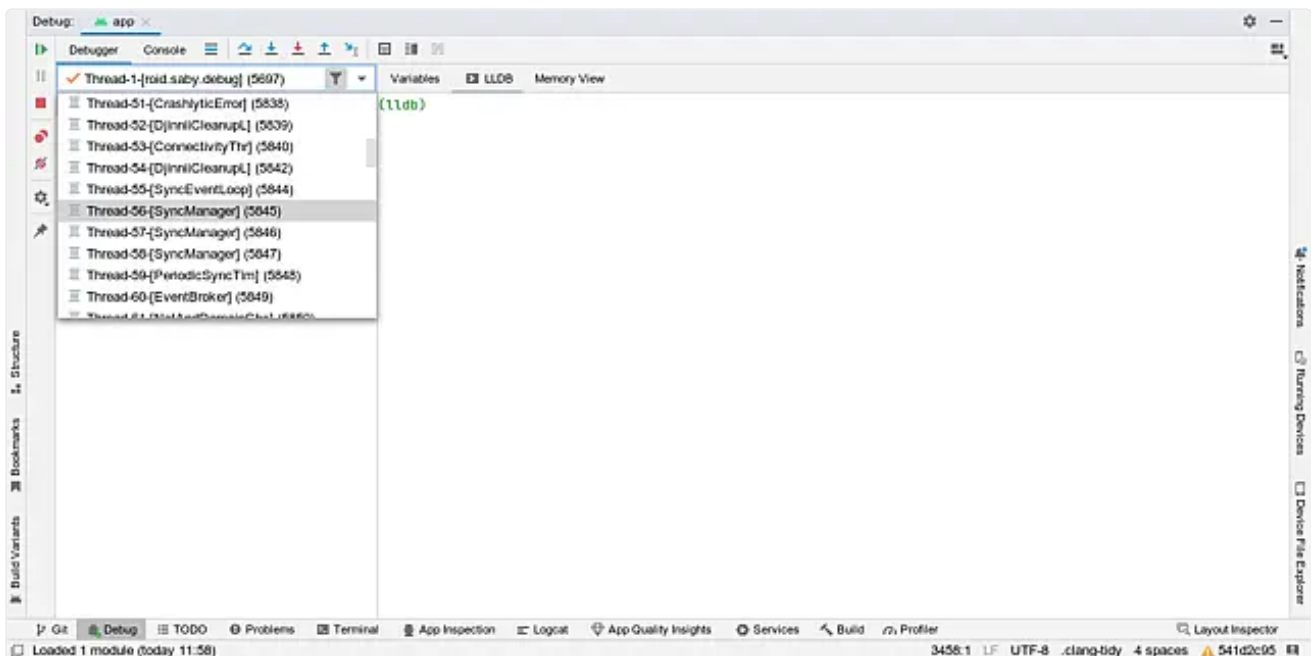
Отладка в LLDB

Здесь описывается отладка корутинызированной синхронизации в мобильных приложениях, а также корутинызированных настольных решений для платформы macOS. Информацию по настройке отладочного окружения можно найти в статьях:

- [Сборка и отладка Android-приложения](#),
- [Сборка и отладка iOS-приложения](#).

Идентификация потока-исполнителя корутин

Как и в случае с GDB, расширение отладки LLDB работает только с корутинами, исполняемыми выбранным потоком. Для настольных решений найти поток планировщика корутин можно аналогично [идентификации потока-исполнителя корутин в GDB](#). В мобильных приложениях следует искать потоки каркаса синхронизации (именованные `SyncManager`).



Перечисление корутин в потоке

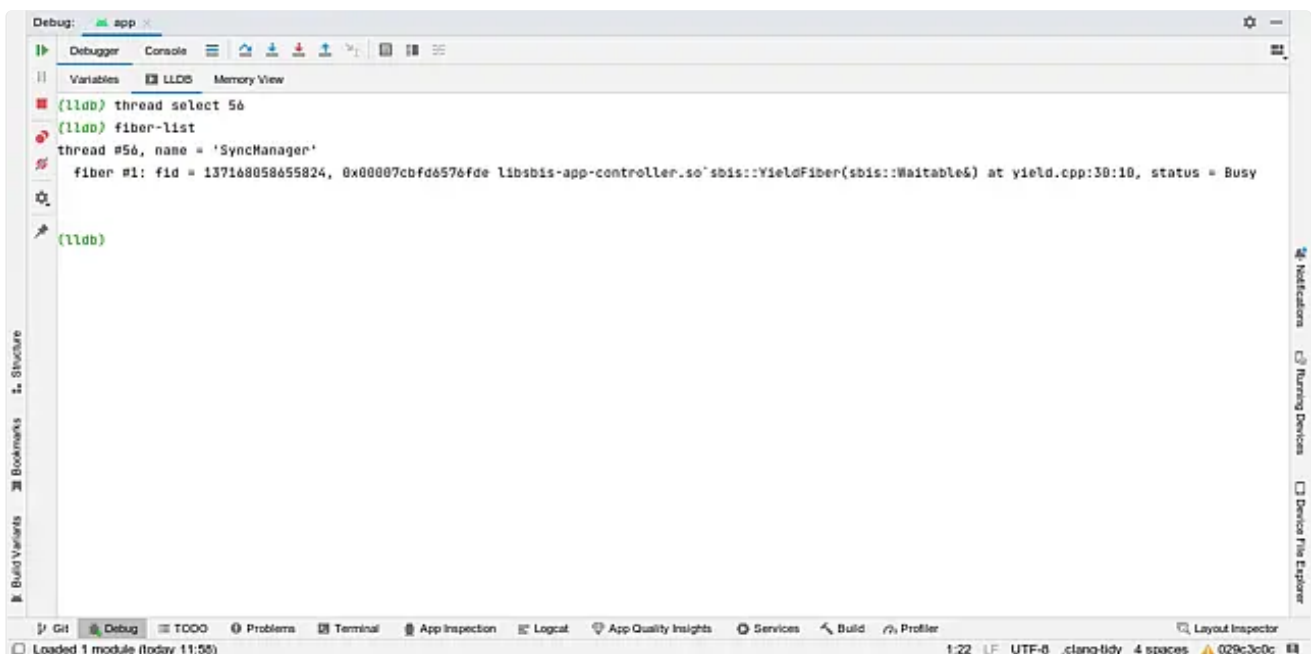
Для вывода корутин, исполняемых в требуемом потоке, необходимо выбрать поток в качестве текущего командой `thread select <индекс-потока>` и вызвать команду `fiber-list`.



В некоторых IDE поток, выбранный в качестве текущего на UI, не является таковым для встроенной консоли отладчика, и/или наоборот.

В консоль будут выведены:

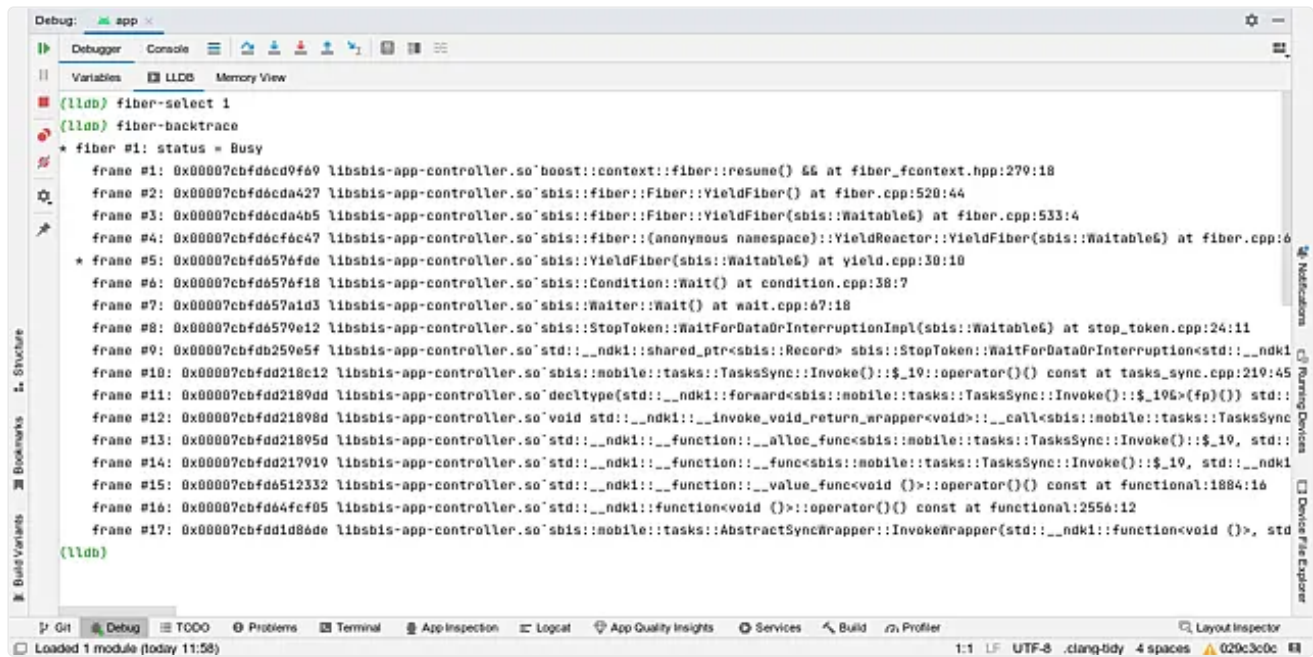
- индекс корутины,
- идентификатор корутины (по совместительству адрес объекта корутины `sbis::fiber::Fiber`),
- актуальная позиция в стеке вызовов (адрес инструкции, имя модуля, имя функции, позиция в исходном коде),
- статус корутины (Active, Busy, Idle).



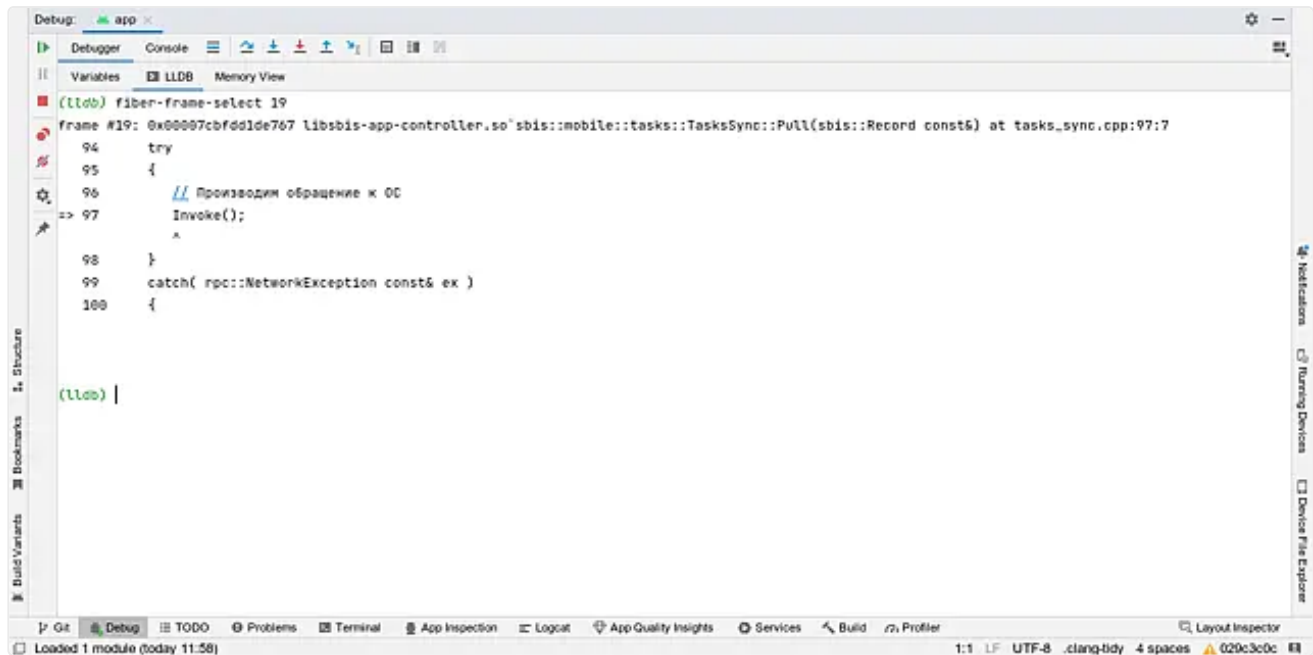
Исследование состояния корутины

Для изучения состояния конкретной корутины требуется выбрать ее в качестве текущей командой `fiber-select <индекс корутины>`, где в качестве индекса корутины указывается одно из значений выданное командой `fiber-list` на предыдущем шаге.

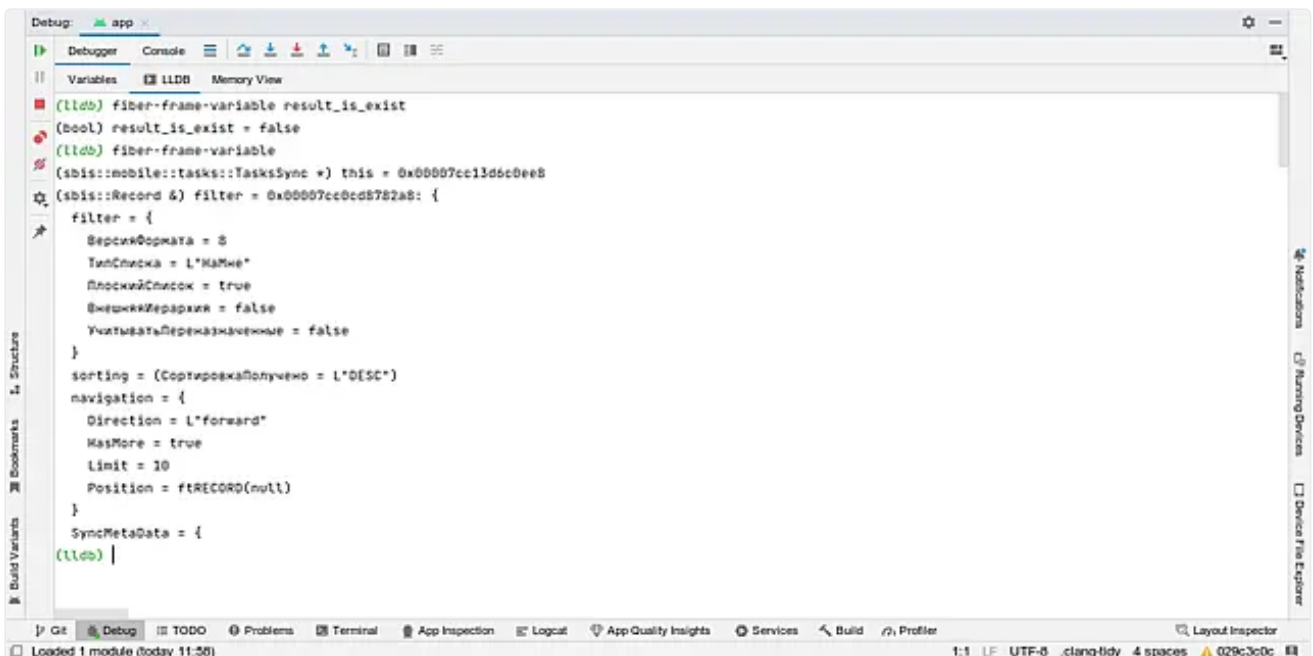
После выбора корутины, ее стек вызовов можно вывести командой fiber-backtrace.



Далее можно выбрать отдельный кадр стека командой `fiber-frame-select <индекс кадра>`. При этом будет показана область кода в окружении текущей исполняемой инструкции.



Значения локальных переменных можно исследовать командой `fiber-frame-variable <имя-переменной> <имя-переменной> [...]`.

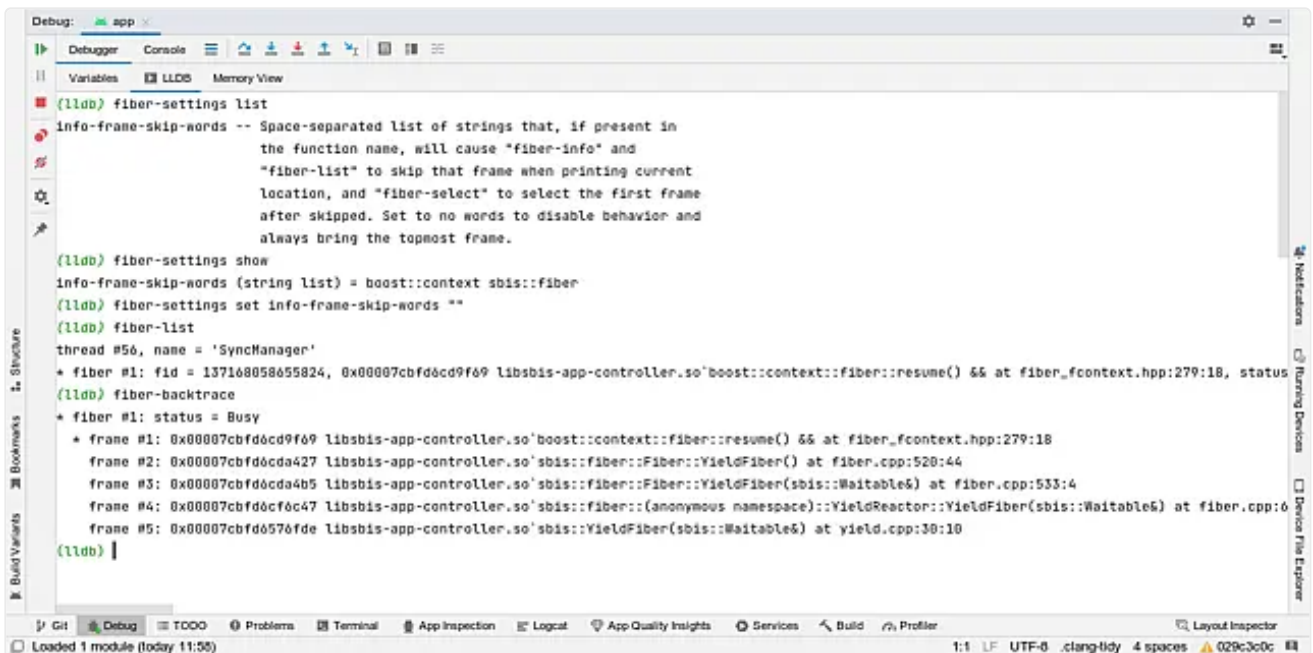


Команда `fiber-frame-variable` без аргументов выводит все переменные текущего блока кода без учета области видимости, поэтому в результате могут встречаться, как переменные, определяемые ниже по коду, так и переменные, чья область видимости уже закончилась.

Вспомогательные команды

Вывод команды `fiber-list` можно подстроить под нужды текущей сессии отладки путем изменения множества ключевых слов, используемых для пропуска неактуальных кадров стека вызовов. По умолчанию с вершины стека отбрасываются кадры пространств имен `boost::context` и `sbis::fiber` и в качестве актуальной позиции выводятся данные первого встреченного кадра не из этих пространств.

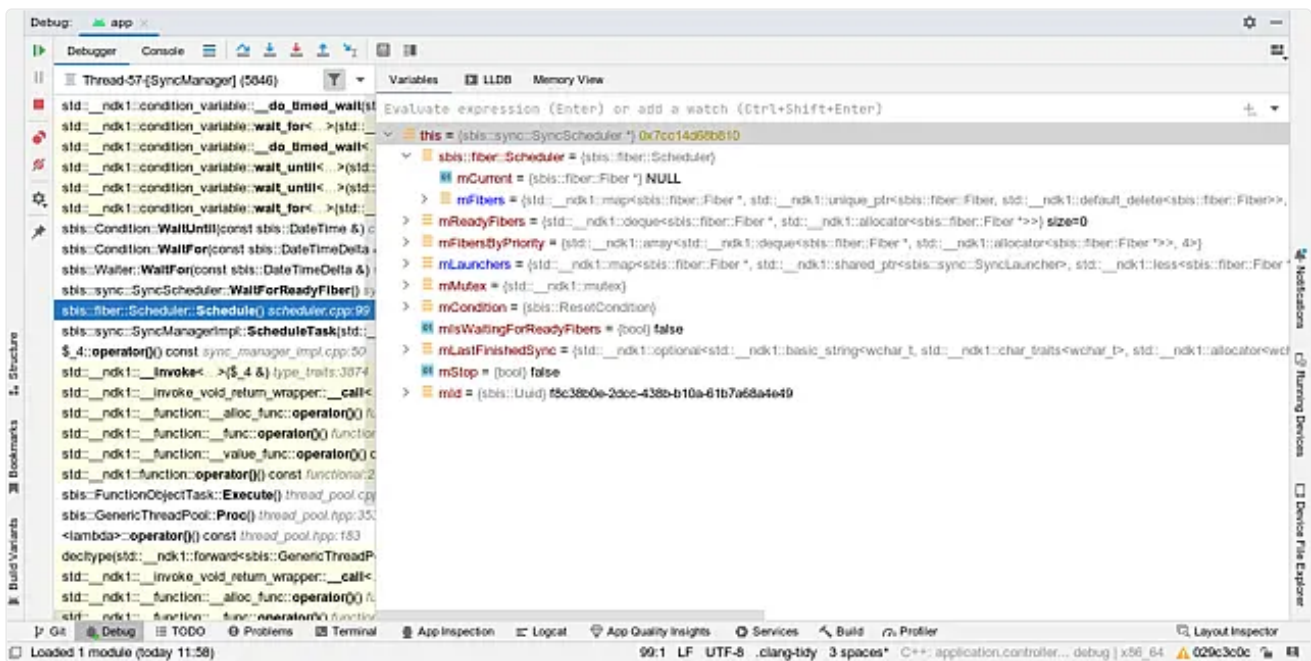
Настройка списка фильтров осуществляется командой `fiber-settings set info-frame-skip-words [<слово> [<слово> [...]]`, где слово -- любая комбинация символов без пробелов. Вывести текущий состав фильтров можно командой `fiber-settings show`.



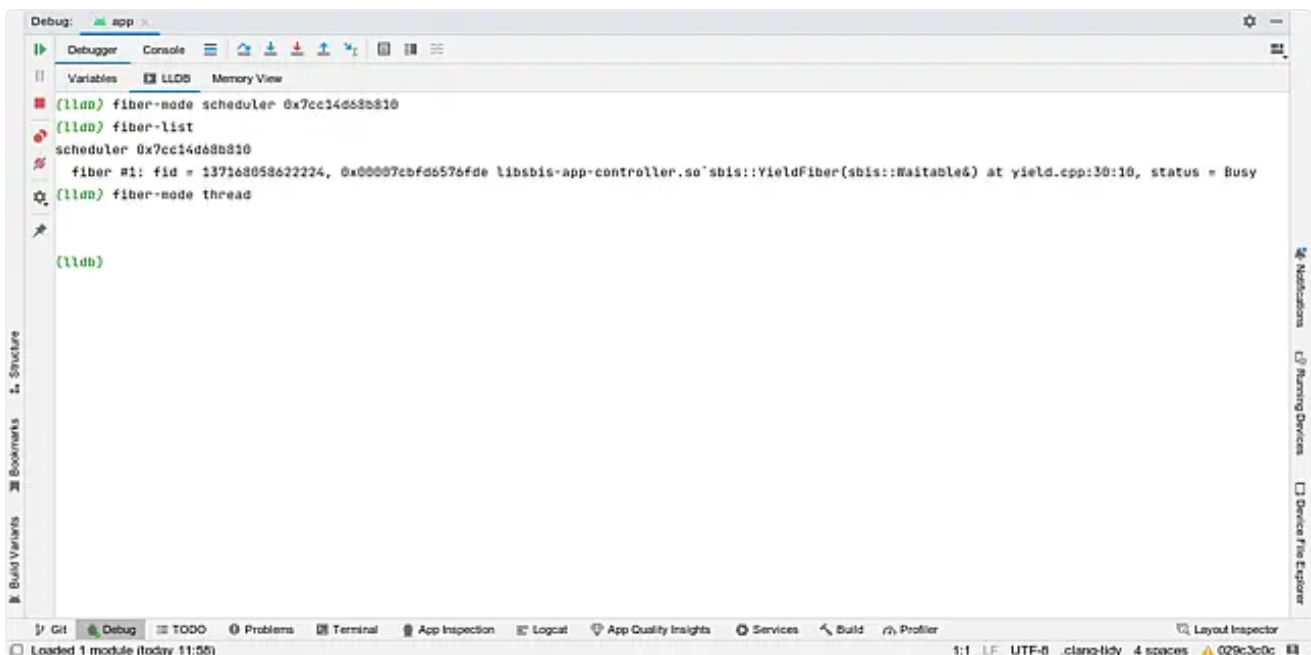
Альтернативный режим перечисления корутин

В дополнение к основному методу перечисления корутин в потоке, который их исполняет, расширение позволяет выводить и работать с корутинами, указывая их родительский планировщик.

Для этого необходимо определить адрес интересующего планировщика.



Затем перевести расширение в режим работы с планировщиком командой `fiber-mode scheduler <адрес-планировщика>`. В этом режиме команда `fiber-list` выведет все корутины, которыми управляет выбранный планировщик, а команды [исследования состояния корутины](#) работают идентично основному режиму.



Для возвращения расширения в режим работы с текущим потоком следует воспользоваться командой `fiber-mode thread`.